

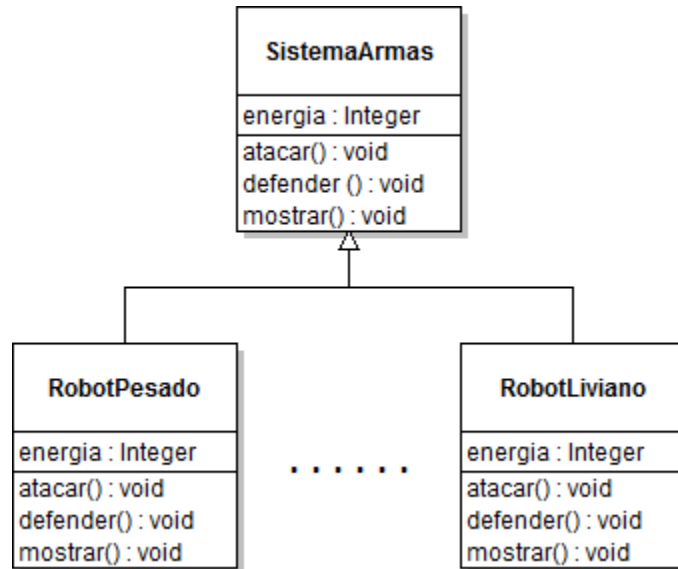
Uso avanzado de las interfaces

Índice

Presentación del caso “Batalla del futuro”	2
Planteo de soluciones.....	3
Los problemas de las soluciones planteadas.....	5
Herencia y sus desventajas.....	7
Principio de diseño 1	7
Separar las familias de comportamientos.....	8
Principio de diseño 2	9
Principio de diseño 3. Favorecer la composición por sobre la herencia.....	12

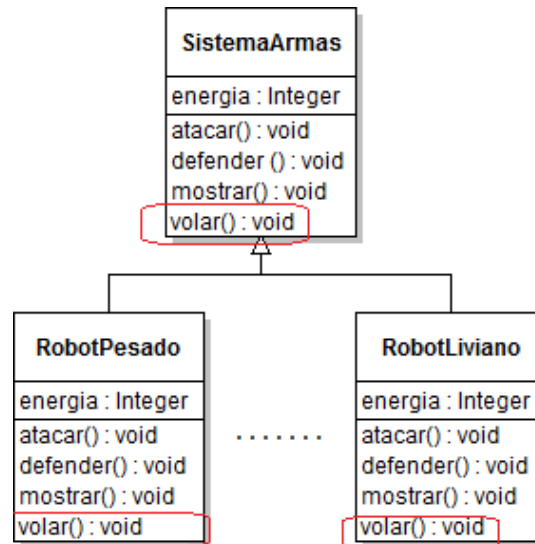
Presentación del caso “Batalla del futuro”

Supongamos que hay que modelar un juego mobile de guerra/estrategia en tiempo real: “Batalla del futuro”. Entre las clases del juego, tendremos los diferentes tipos y sus características. El primer paso es definir un robot con sus operaciones básicas:



Lo primero que podemos hacer es crear una clase **SistemaArmas**, con las operaciones comunes: atacar, defender y mostrarse (en pantalla, con sus datos). Supongamos que una próxima actualización del juego introducirá sistemas de armas voladores. Sería importante que se mantengan las “actualizaciones” de la app al mínimo y que se detenga lo menos posible el sistema. Entonces, ¿1ué podría hacerse con el diseño actual?

Normalmente se incluiría la operación `volar()` en la clase **SistemaArmas**. Pero ¿qué pasa si entre los sistemas de armas que quiero incluir hay un tanque? ¡Los tanques no vuelan! Veamos un ejemplo:

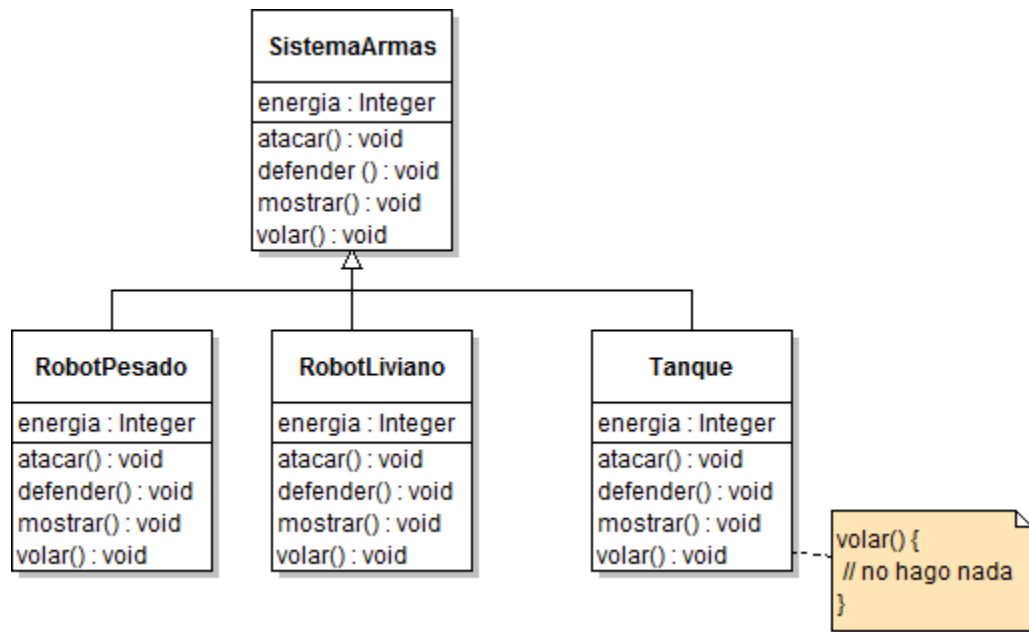


A simple vista, un sistema de armas puede ser un robot, un tanque, un bombardero... entonces, agregando el método `volar()` se rompe el diseño, porque de existir una clase hija tanque, tendríamos un tanque volador. Entonces, ¡No todos los vehículos deberían volar! No es un buen diseño. Ni siquiera el diseño original era bueno, dado que un cambio local a una clase generó un efecto colateral generalizado.

Entonces, en lo que a mantenimiento respecta, la herencia no siempre es la mejor opción.

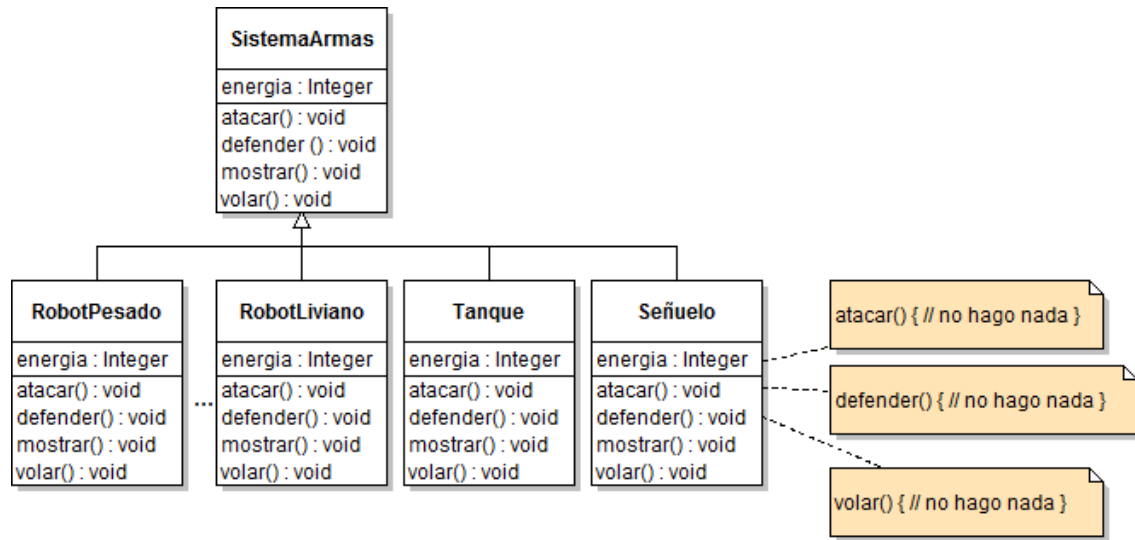
Planteo de soluciones

Entonces, volvemos al problema del zoo virtual. Qué tal si sobrescribimos la operación `volar()` y la dejamos “vacía” (sin implementación) en todas aquellas clases “que no deban volar” (tanque, submarino, etc.).



Si sobrescribimos el `volar` para dejarle una implementación vacía, ¿qué pasa si agrego un nuevo tipo de vehículo, por ejemplo, un portaaviones? Un barco no debería volar: otra vez, tengo que dejar una implementación vacía.

Para empeorar las cosas, ¿qué pasa si agrego un `VehículoSeñuelo`? Ese tampoco debería “volar” y no queremos seguir agregando implementaciones vacías. Atención con este caso en particular, porque tampoco debería atacar ni defenderse. De ser así, no se puede empezar a dejar implementaciones vacías por todos lados (ya vimos que esta no es la solución correcta, por varias razones). Además, estaríamos duplicando el código por todos lados y dejamos un software susceptible a errores.



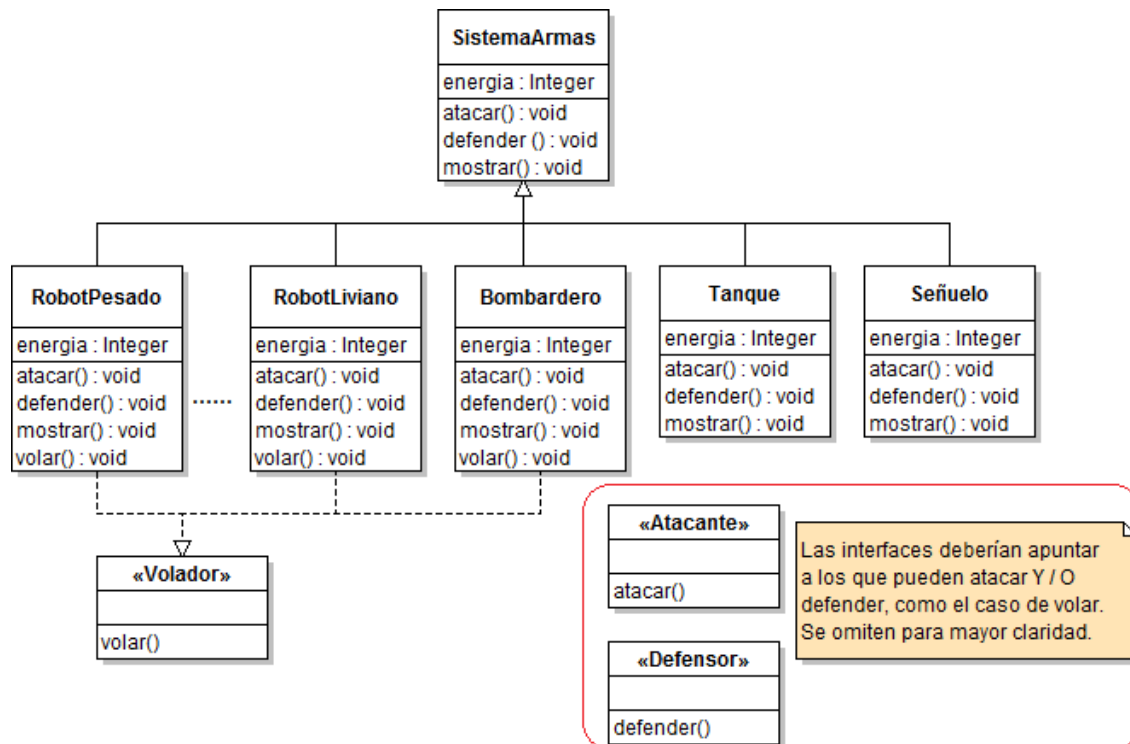
Podríamos hacer clases abstractas, en las cuales tengamos implantaciones vacías por defecto. Aunque, si necesitamos una clase que tenga diferentes combinaciones de `atacar()`, `defender()`, `volar()`, `sumergirse()` etc. perderíamos mucho en lo que a polimorfismo respecta.

Los problemas de las soluciones planteadas

Con estos aspectos a mano podemos ver qué desventajas tiene la herencia para establecer el comportamiento de un Sistema De Armas.

- Se duplica código en las hijas.
- Un cambio sencillo podría afectar todo el modelo.
- Cambiar el comportamiento de los vehículos en *runtime* es casi imposible.

¿Qué otra construcción vimos que podría solucionar este problema? Las interfaces.



Hasta aquí, solucionamos parte del problema, como habíamos hecho con el zoo virtual. Pero si en algún momento quisiéramos cambiar la forma en que vuela cada sistema de armas volador, habría que revisar cada uno de los métodos y hacerle los cambios necesarios. Qué pasaría si hubiera 20, 30 sistemas de armas diferentes... Recordemos que queremos minimizar los impactos en el código para no andar liberando nuevas versiones y que nuestros usuarios tengan que bajarse de la tienda actualizaciones del juego.

Entonces, hasta aquí las interfaces solo resolvieron una parte del problema, es decir, ya no habrá tanques voladores (o señuelos que ataquen y se defiendan). Pero no podemos reutilizar código ni aprovechar al máximo el polimorfismo (ya vimos las deficiencias del simple uso de interfaces en estos problemas con el zoo virtual). De hecho, no podemos usar SistemaArmas como tipo de nuestros objetos → mejoramos una cosa, empeoramos otra.

Conclusión: no siempre la herencia y/o las interfaces solucionan todos los problemas.

Herencia y sus desventajas

Revisando el caso de los “Sistemas de Armas” vimos que, como el comportamiento de cada sistema de armas varía en cada subclase y algunos “subsistemas” de armas ni siquiera deberían tener algunos del comportamiento. Entonces, podemos afirmar que la herencia no resuelve completamente el problema.

Usar interfaces tampoco resuelve el problema, ya que, al menos en Java, las operaciones definidas en las interfaces no pueden llevar código (en las últimas versiones esto está cambiando, pero eso escapa al scope de esta materia). Entonces, no se puede reusar el comportamiento a través de todos los sistemas que tienen una misma manera de atacar() o de volar().

En lo que a software se refiere, el cambio es continuo. Siempre hay nuevos requerimientos, nuevas necesidades, nuevas regulaciones, etc. Lo mejor para ahorrarse problemas a futuro es “encerrar” o “englobar” aquellos aspectos que cambien, para que al cambiar no afecten el resto del sistema. Es decir, conviene encapsular todo lo que cambie y aislarlo de lo que es fijo a lo largo del tiempo. Así, se logrará que los cambios no afecten de forma inesperada a otras partes del sistema y se logrará mayor flexibilidad.

Principio de diseño 1

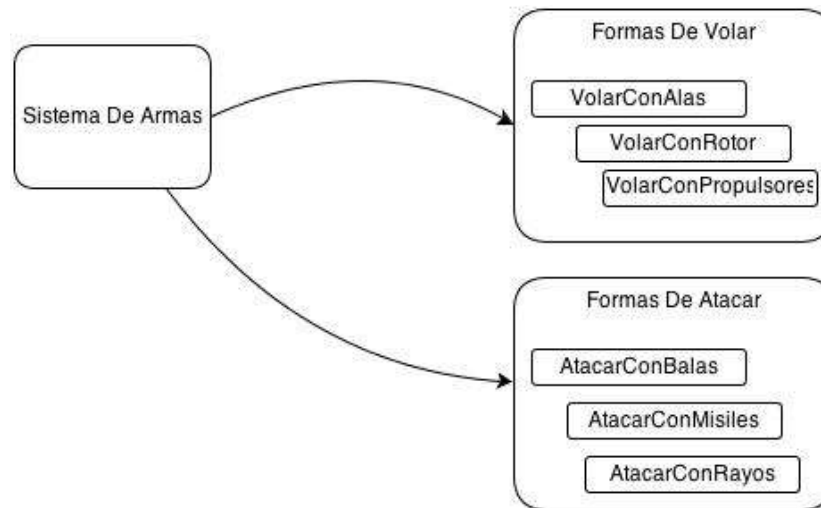
Hay un principio de diseño de software que alienta a identificar los aspectos de nuestra aplicación que sean cambiantes y separarlos de aquello que queda siempre fijo.

Otra forma de decirlo sería: hay que agarrar las partes de un sistema que varían y encapsularlas para que luego se puedan extender o cambiar sin afectar a las partes que no varían.

Este principio forma la base de casi todos los patrones de diseño: hacer que una parte del sistema varíe independientemente de las otras partes.

Volviendo al caso de la “Batalla del futuro”, sabemos que los comportamientos atacar() y volar() cambian de un SistemaDeArmas a otro. Mientras que el mostrarse() o el defender() son siempre los mismos: uno muestra los datos en pantalla y el otro se interpone entre un

enemigo y su objetivo. Entonces, habría que hacer a un lado estos dos comportamientos y englobarlos en pequeñas “familias” de comportamientos similares. Es decir, deberíamos tener algo que agrupe las diferentes formas de atacar() y las diferentes formas de volar(). Entonces, como el atributo “energía” y las operaciones mostrarse() y defender() están bien. Dejaremos la clase “SistemaArmas” como está. Solo sacaremos las operaciones de volar() y atacar().



Separar las familias de comportamientos

De tratarse de un juego de guerra, cada misión tendría diferentes objetivos. Por lo que un sistema de armas en particular tendría que poder cambiar su forma de atacar() si se tratara de un objetivo en tierra o en el aire o en el mar. Es decir, tendríamos que poder “asignar” una forma de atacar(). Otro posible cambio es poder cambiar la forma de atacar() si el usuario paga un adicional o si se suscribe al pago mensual por ejemplo.

Dado que se trata de una batalla del futuro, algunos de los sistemas de armas disponibles podrían “transformarse” y convertirse de un avión a un robot que ataca por tierra. Entonces, estaría bueno poder “cambiar” el comportamiento de atacar() y volar() estando ya el sistema de armas instanciado y corriendo. Es decir, tendríamos que asignarle la forma de atacar() y volar() en *runtime*. Es decir, sin parar el sistema, sin cambiar el código, sin liberar una nueva versión, yo quiero poder cambiar el comportamiento. Si el usuario

pagó un extra, le puedo cambiar la forma de volar de sus aviones o que el atacar() sea con doble daño.

Hecho esto último, cada forma de atacar o volar ya no vendrá dictada por el sistema de armas del cual se trate. Sino que el sistema de armas tiene una forma de atacar o volar. De poner la forma concreta de atacar o volar estaríamos acoplando una “forma de” con un sistema de armas en particular. Esto resolvería solo en parte el problema. Entonces, necesitamos introducir una interfaz que dictamine de la “forma de” volar() y atacar() para que ya sea un avión, un tanque o un robot, cuando se le ordene volar o atacar este lo hará y sin saber los detalles de implementación de cada comportamiento (quien dice, puedo ofrecerle a mis usuarios, que en el especial de navidad, si paga 50 dólares, los tanques pueden volar y lanzar gorritos de navidad...) => hemos ENCAPSULADO las formas de volar() y atacar().

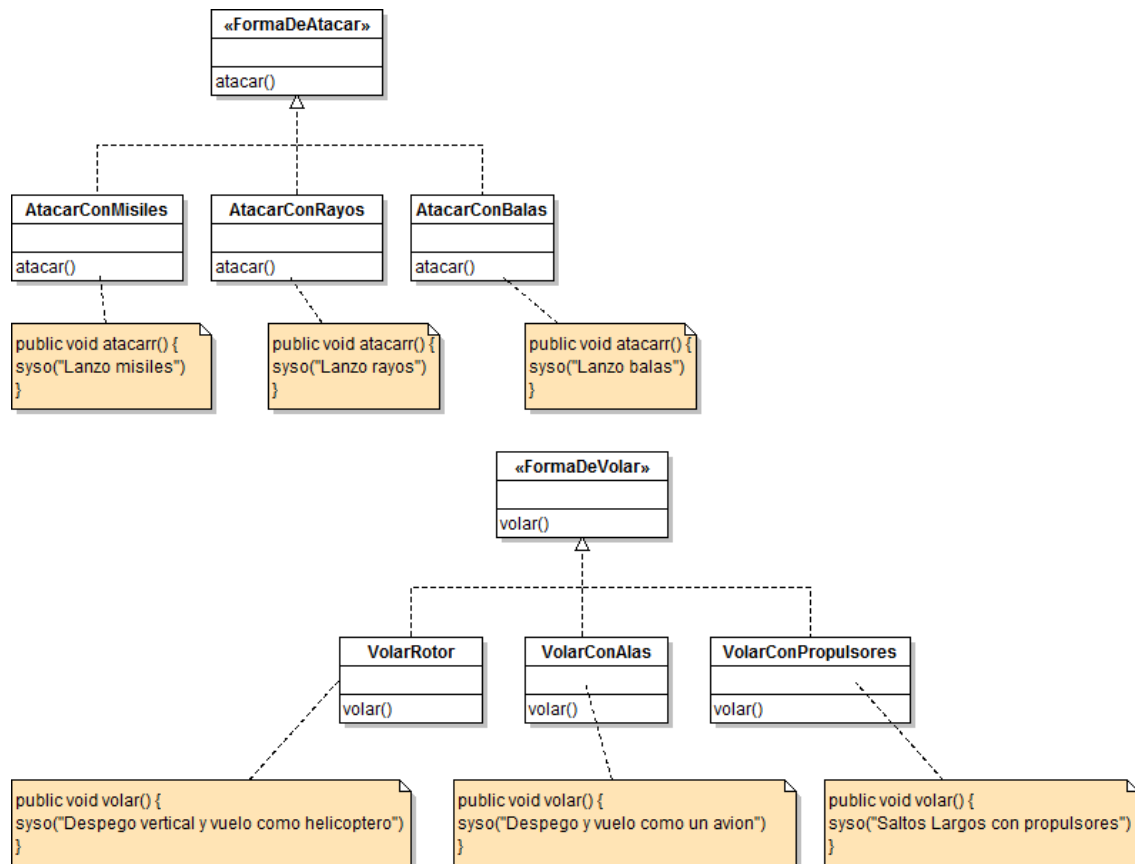
Principio de diseño 2

Podemos dar paso a un segundo principio de diseño, que dice que debe programarse contra una interfaz y no contra una implementación. Es decir, del lado izquierdo del igual, siempre debemos tener el tipo de la interfaz, y del lado derecho sí podremos usar la implementación que deseemos.

Desde este punto, la clase SistemaDeArmas ya no será quien implemente los comportamientos de volar() y atacar(). Habrá una serie de clases cuyo solo propósito sea implementar los diferentes comportamientos.

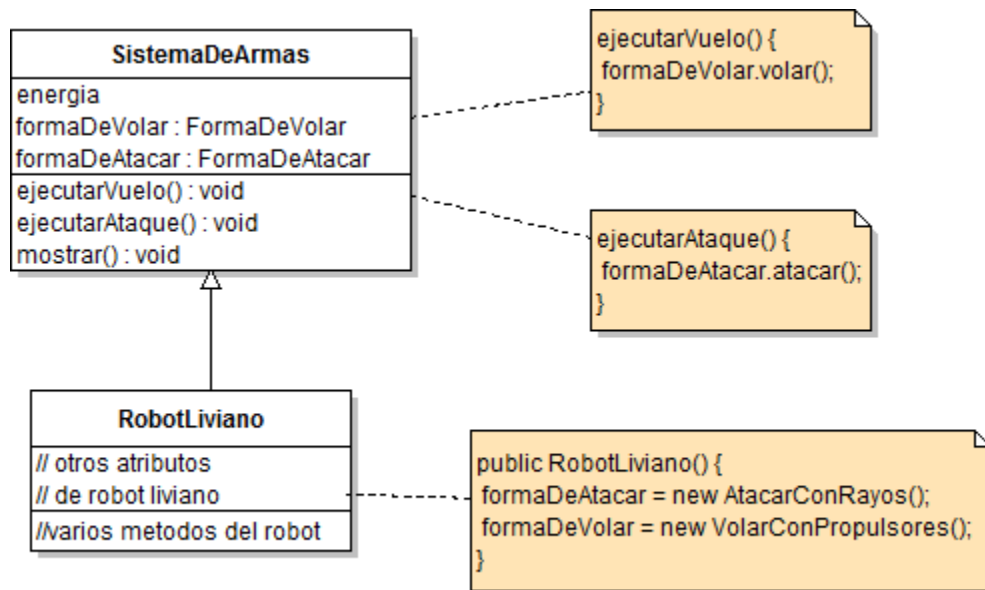
Antes, lo que hacíamos era o heredar directamente el comportamiento desde de la clase SistemaDeArmas o especificarlo en alguna de sus hijas. En ambos casos, dependíamos de una implementación. De esta manera, no se podía alterar el comportamiento sin alterar el código.

Ahora, de lo que disponemos es una interfaz que define un comportamiento y serán las sucesivas y diferentes implementaciones de estas que dicten cómo se lleva a cabo. ¡A partir de ahora podemos alterar el comportamiento sin tocar código!



Un efecto secundario de este *approach* es el hecho de que las clases que implementan los diferentes comportamientos ahora pueden ser reutilizadas en otros escenarios. Por ejemplo, si decidiéramos actualizar nuestro juego y que las batallas sucedan también en el espacio, algunas “formas de atacar” podríamos utilizarlas nuevamente. Sin mencionar que se pueden agregar nuevas formas sin afectar el diseño ni el código existente.

Veamos cómo quedaría el diseño:



Las formas de atacar y volar son ahora referencias a clases que tendrán la responsabilidad de atacar y volar según corresponda. En vez de que el **SistemaDeArmas** sea quien maneje el ataque y el vuelo, estos comportamientos se delegan al objeto referenciado por el atributo “`formaDeVolar`”. Hasta aquí no importa qué tipo de objeto sea, solo importa que ese objeto sabe cómo `volar()` de la forma correspondiente. Pasa lo mismo con el ataque.

Entonces, tiempo de ejecución, cuando sobre **RobotLiviano**, se ejecute el método heredado `ejecutarAtaque()`, se mostrará por consola el mensaje “Lanzo rayos”. Asimismo, si llamo al setter de `formaDeAtaque` y se cambia la forma por un **AtacarConMisiles**, el mensaje pasa a ser “Lanzo Misiles”.

MAIN:

```

public class Guerra {
    public static void main(String[] args) {
        SistemaDeArmas miRobot = new RobotLiviano();
        miRobot.ejecutarVuelo();
        miRobot.ejecutarAtaque();

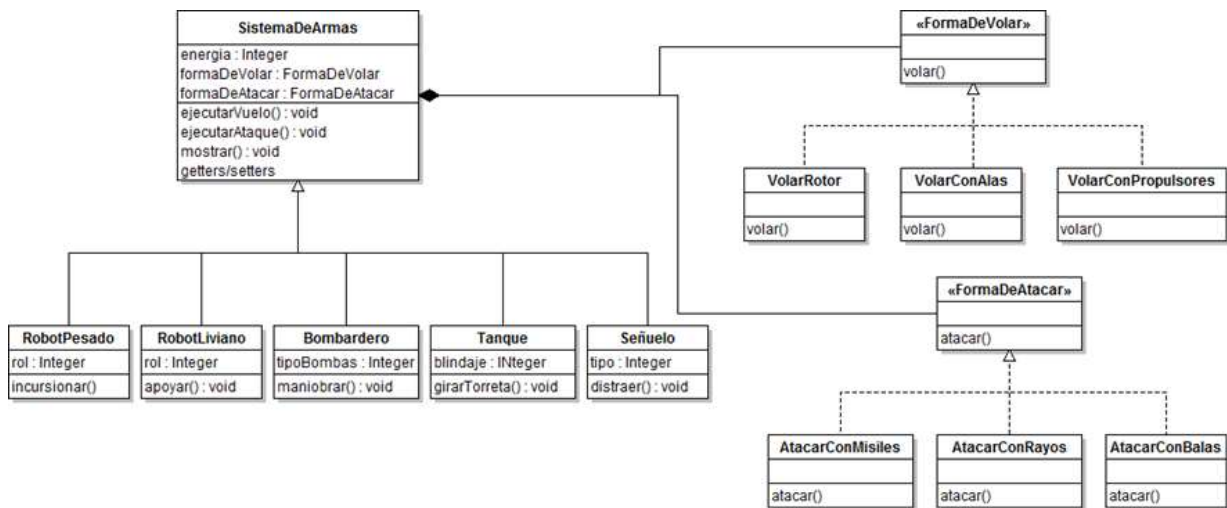
        miRobot.setFormaDeAtacar(new AtacarConMisiles());
        miRobot.ejecutarAtaque();
    }
}
  
```

Consola:

```
Salto Largo con Propulsores
Lanzo Rayos
Lanzo Misiles
```

Principio de diseño 3. Favorecer la composición por sobre la herencia

Si vemos todo el panorama completo, veríamos algo así:



Si generalizamos el problema, podemos pensar en las diferentes “formas de” como diferentes “algoritmos” o “familias de algoritmos”. Es decir, cada “algoritmo” representa algo que un sistema de armas puede hacer.

Componer nos da mayor flexibilidad. Heredar nos “encerraba” en comportamientos fijos. Componer nos permite encapsular los comportamientos, permitiendo intercambiarlas sin afectar otras partes del sistema y más aún: nos permite cambiar los comportamientos en RUNTIME (siempre que se respeten las interfaces). Esto da lugar a otro principio de diseño muy importante: favorecer la composición por sobre la herencia.